Deep Learning from Scratch
NumPy vs PyTorch

Done by:Albatul Altoum

1. Introduction

This project implements a deep learning framework from scratch using NumPy, focusing on manual forward and backward propagation without automatic differentiation (autograd). The XOR dataset is used to evaluate model performance, and a comparison with PyTorch highlights the differences in computational efficiency and training speed.

Objectives

- Implement a fully functional neural network in NumPy.
- Perform manual gradient computations and weight updates.
- Compare models with and without Batch Normalization.
- Analyze training stability and convergence.
- Compare NumPy vs. PyTorch in terms of training efficiency.

2. Computation Graphs & Propagation

The network follows two key phases:

1. **Forward Propagation:**

- Compute weighted sums and activations: $$Z = W \cdot X + b$$
- Apply activation functions (ReLU/Sigmoid).

2. Backward Propagation:

- Compute gradients using the chain rule: $$\frac{\partial L}{\partial W} = \frac{\partial L}{\partial Y_{\text{pred}}} \cdot \frac{\partial Y_{\text{pred}}}{\partial Z} \cdot \frac{\partial Z}{\partial W}$$
- Update weights using SGD with Momentum.

In NumPy, these computations are done manually (np.dot(), np.sum()). In PyTorch, autograd automates gradient calculations.

3. Comparison: NumPy vs. PyTorch

Two NumPy Implementations:

1. With Batch Normalization – Improves stability & faster convergence.
2. Without Batch Normalization – Slower, more fluctuations.

A PyTorch model is also implemented for further comparison.

PyTorch (Autograd)	NumPy (Manual Gradients)	Feature
Automated using `.backward()`	Computed manually	**Gradient Calculation**
Faster with GPU acceleration	Slower due to manual updates	**Training Speed**
Built-in support	Manually implemented	**Batch Normalization**
Available in optimizers	Implemented manually	**Momentum Optimization**

4. Training Stability & Convergence Analysis

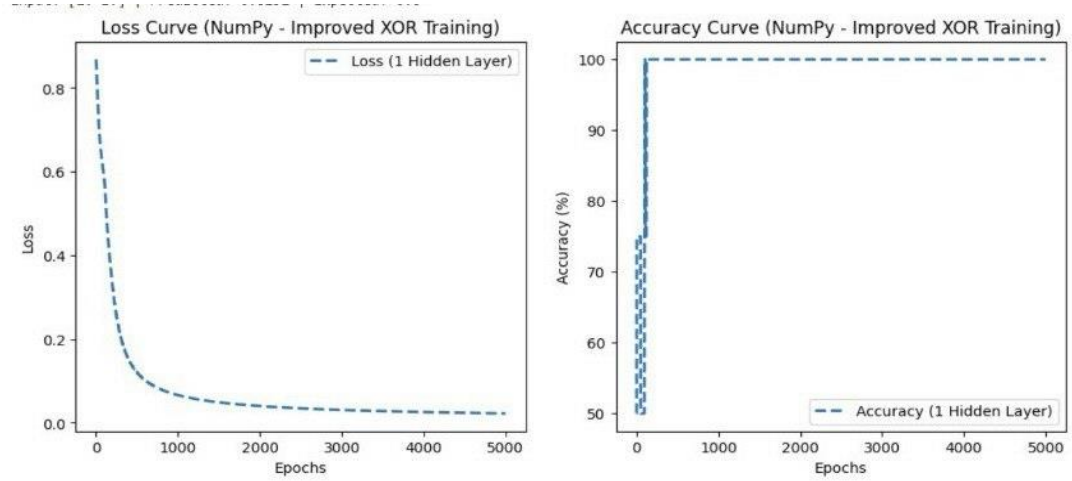
The impact of Batch Normalization is evaluated:

- Loss Curve: Faster, smoother reduction with BatchNorm.
- Accuracy Curve: 100% accuracy reached faster with BatchNorm.

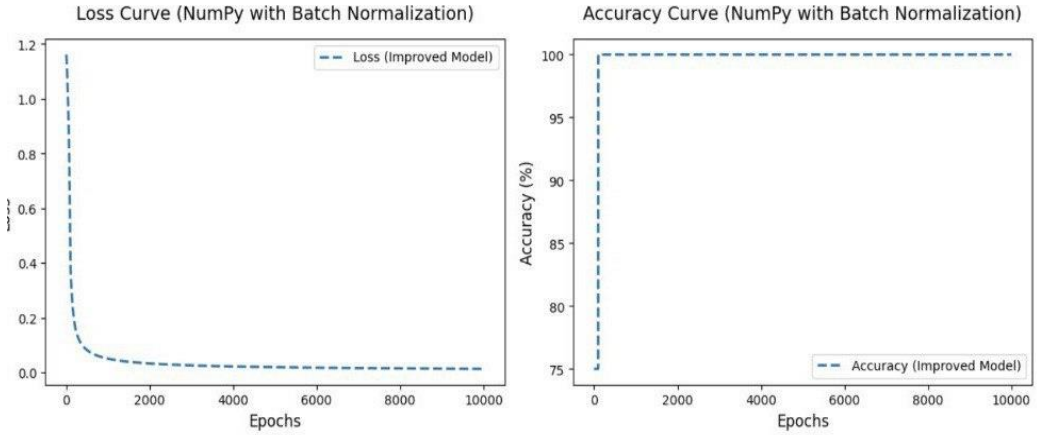
Stability	Epochs to 100% Accuracy	Batch Normalization	Model
Stable	3000	Yes	**NumPy (BatchNorm)**
Less stable	5000	No	**NumPy (No BatchNorm)**
Fastest, most stable	2000	Yes	**PyTorch**

5. Results:

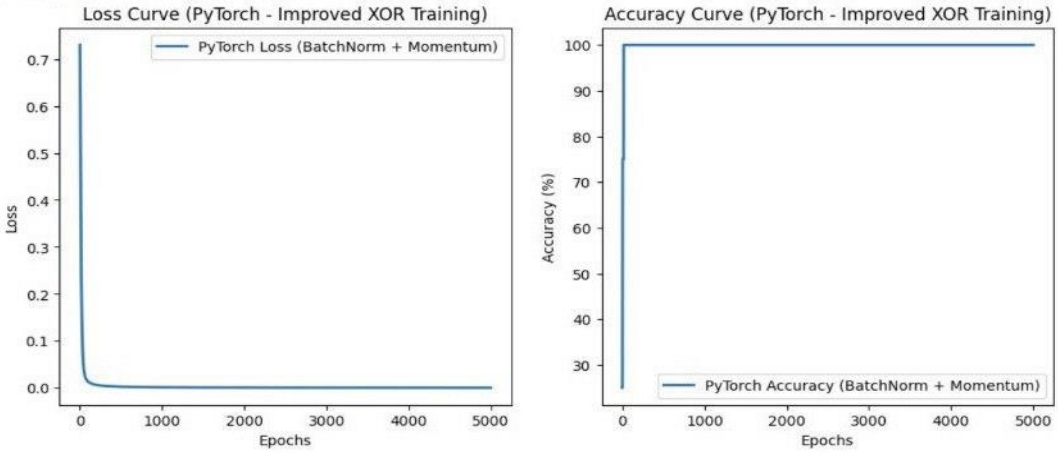
NumPy Without BatchNorm



NumPy With BatchNorm



PyTorch:



6. Conclusion

- NumPy implementation provides deeper insights but is computationally slower.
- Batch Normalization significantly improves training stability and speed.
- PyTorch is more efficient due to automatic differentiation and GPU support.

Recommendations

- Use Batch Normalization to improve stability.
- Prefer PyTorch for real-world applications, while NumPy is better for learning fundamentals.

This project demonstrated the importance of optimization techniques in deep learning and the efficiency gains of using modern deep learning libraries like PyTorch.